

Tutorial 04

▼ Overview

Tutorial

- ADT
- Binary Search Tree

Lab

- Binary Search Trees (Covered in Tutorial)
- Level-Order Traversal (Briefly Mentioned in Tutorial)
- Complexity Analysis Practice (Week 2 Tutorial / Lectures)

1 Mark handmarking on complexity analysis.

▼ Major Announcements

- Assessment 1 out this week Wednesday, due 8pm Monday Week 7
- Get it done early
 - Use good style (any style questions ask, use style guide)
 - Use good analysis
- Any questions on the assignment head to help sessions / forum on the course website

▼ ADT

▼ What does ADT stand for? What is an ADT?

- Abstract Data Type
 - Abstract is implying that you will not need to know the implementation in order to use it.
 - A data type is anything from an int to a queue or a tree. What this therefore means is that you can use the data type and do not need to know how it is implemented, which is abstraction.

▼ What is a stack?

Last In / First Out

Container that holds elements and is Last In / First Out

- e.g. pile of pancakes, deck of cards on a table.

▼ What is a queue?

A container that is First In / First Out,

- e.g any line to wait for something, single lane one way road etc.



Use the stack ADT interface to complete the implementation of the queue ADT below?

▼ What are the operations on a stack

- Pushing to the stack, adding an element.
- Popping removing an element for your use assumes not empty.
- Notice: we are using int's as items in the stack
 - this can be any type for ADTs and is often labelled item instead of int to show that it is built for any item
- Reminder: Last in, First Out

Stack ADT:

```
// Creates a new empty stack
Stack StackNew(void);

// Frees all memory allocated to the stack
void StackFree(Stack s);

// Pushes an item onto the stack
void StackPush(Stack s, int item);

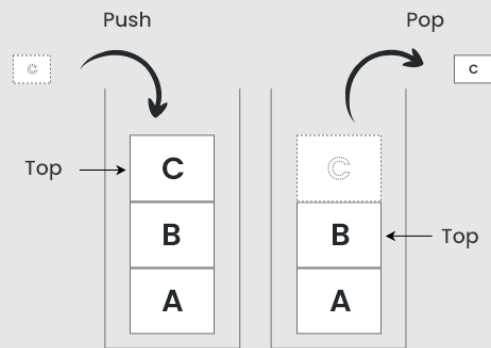
// Pops an item from the stack and returns it
// Assumes that the stack is not empty
int StackPop(Stack s);

// Returns the number of items on the stack
int StackSize(Stack s);
```



Stack

Data Structure



From GeeksForGeeks Website

▼ Queue

Common computer science problems of implementing a queue using two stacks.

- Queues are first in first out.

```
#include "Stack.h"

struct queue {
    Stack s1;
    Stack s2;
};

Queue QueueNew(void)
{
    Queue q = malloc(
        sizeof(struct queue));
    q->s1 = StackNew(
    );
    q->s2 = StackNew(
    );
    return q;
}

void QueueFree(Queue
q) {
    StackFree(q->s1);
    StackFree(q->s2);
    free(q);
}

void QueueEnqueue(Que
ue q, int item) {
    // TODO
}
```

```
// Creates a new empty stack
Stack StackNew(void);

// Frees all memory allocated to the stack
void StackFree(Stack
s);

// Pushes an item onto the stack
void StackPush(Stack
s, int item);

// Pops an item from the stack and returns it
// Assumes that the stack is not empty
int StackPop(Stack
s);

// Returns the number of items on the stack
int StackSize(Stack
s);
```

```
int QueueDequeue(Queue q) {  
    // TO DO  
}
```



Code Queue enqueue Using Stack

```
void QueueEnqueue(Queue q, int item) {  
    StackPush(q->s,  
1, item);  
}
```



Code Queue dequeue (inefficient & efficient) (Hints inside) (Table Groups)

▼ Hint (inefficient $O(n)$)

Process is you want to note that a stack is first in last out and a queue is first in first out so you need to switch the order of the stack. This can be done by moving all elements to the second stack getting now the last item inserted which is the first inserted into the first stack and redoing this process.

▼ Answer

```
int QueueDequeue
(Queue q) {
    // Unqueues
    all the elements
    in stack 1 movin
    g to stack 2.
    while (Stack
    Size(q->s1) > 0)
    {
        int item
        = StackPop(q->s
        1);
        StackPus
        h(q->s2, item);
    }

    // Gets the
    item
    int dequeued
    Item = StackPop
    (q->s2);

    // Moves all
    elements back to
    stack 1 to resta
    rt process.
    while (Stack
    Size(q->s2) > 0)
    {
        int item
        = StackPop(q->s
        2);
        StackPus
        h(q->s1, item);
    }
```

```

        // Returns i
tem
        return deque
uedItem;
}

```

▼ Hint (more efficient)

Only move what you need, each time. This is in the form of if the second stack is empty then move all new elements to the second stack.

- Maintains order first in first out but means if you insert a lot of elements you don't have to move again until your second stack empties.

Here is a more visual example.

Stacks	Enqueue 1,2,3,4,5,6
s1	1,2,3,4,5,6
s2	
Return	N/A

▼ Answer (More Efficient)

```
int QueueDequeue
(Queue q) {
    if (StackSize(q->s2) == 0) {
        while (StackSize(q->s1) > 0) {
            int
            item = StackPop
            (q->s1);
            StackPush(q->s2, item);
        }
    }

    int dequeued
    Item = StackPop
    (q->s2);
    return dequeuedItem;
}
```

▼ Binary Search Tree

Data Type

```
struct node {  
    int value;  
    struct node *left;  
    struct node *right;  
}
```

▼ Tree Insertion: For each of the sequences below

- start from an initially empty binary search tree
- show the tree resulting from inserting values in the order given

1. 4 6 5 2 1 3 7 2. 5 2 3 4 6 1 7 3. 7 6 5 4 3 2 1

- Node
- Attached to
- Left or Right

▼ Visualiser For Examples (Not Screen)

<https://see-algorithms.com/data-structures/BST>

<https://see-algorithms.com/data-structures/BST>

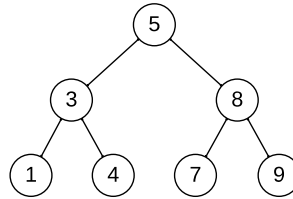
▼ What order can we display tree values?



Tree with different output ordering

What is in-order?

Print Left then
root then right.



In-order	1 3 4 5 7 8 9
Pre-order	5 3 1 4 8 7 9
Post-order	1 4 3 7 9 8 5
Level-order	5 3 8 1 4 7 9



What is pre-order?

Print root then
left then right.



What is post-order?

Print left then
right then root.



What is level-order?

Print the root
then it's children,
then their
children.



What kind of trees have the property that their in-order traversal is the same as their pre-order traversal? Answer Inside

- Right-Deep Trees (No left nodes) inserting ascending order (linked list essentially)



Are there any kinds of trees for which all output orders will be the same? Answer Inside

Empty trees and trees with one node.



Write a recursive function to count the total number of nodes in a tree.

```
int bstNumNodes(struct node *t) {  
    // Enter Tree Here Base Cases  
  
    // Recursive Case  
}
```

▼ Answer

```
int bstNumNodes(structnode *t) {  
    if (t == NULL) {  
        return 0;  
    } else {  
        return 1 + bstNumNodes(t->left) + bstNumNodes(t->right);  
    }  
}
```

Table Group Activity

```
// Counts number of odds  
int bstCountOdds(struct node *t) {}  
  
// Counts number of nodes with at least one child  
int bstCountInternal(struct node *t) {}  
  
// Counts height of a tree, the path with the longest length  
// Path length is a count of the number of (edges) (An empty tree has -1 one node has 0 so on)  
int bstHeight(struct node *t) {}
```

```
// Returns the level of a node given the key. (Can be done iterative or recursive)
// Level of the root is 0 any children have level 1 etc.
int bstNodeLevel(struct node *t, int key) {}

// Returns the amount of nodes that are greater than a given value
int bstCountGreater(struct node *t, int val) {}
```



Implement the following function that counts the number of odd values in a tree.

```
int bstCountOdds(struct node *t) {
    // Base Case

    // Recursive Case

}
```

▼ Answer

```
int bstCountOdds(struct node *t) {
    if (t == NULL) {
        return 0;
    } else if (t->value % 2 != 0) {
        return 1 + bstCountOdds(t->left) + bstCountOdds(t->right);
    } else {
        return bstCountOdds(t->left) + bstCountOdds(t->right);
    }
}
```



Implement the following function to count number of internal nodes in a given tree.

- Internal nodes have at least one child node.

```
int bstCountInternal(struct node *t) {  
    // Base Case  
  
    // Recursive Case  
}
```

▼ Answer

```
int bstCountInternal(struct node *t) {  
    if (t == NULL) {  
        return 0;  
    } else if (t->left == NULL && t->right == NULL)  
    {  
        return 0;  
    } else {  
        return 1 + bstCountInternal(t->left) + bstC  
ountInternal(t->right);  
    }  
}
```



Write a recursive function to compute the height of a tree.

The *height* of a tree is defined as the length of the longest path from the root to a leaf.

The *path length* is a count of the number of *links* (edges) on the path.

The height of an empty tree is -1.

```
int bstHeight(struct node *t) { ... }
```

▼ Answer

```
int bstHeight(structnode *t) {  
    if (t == NULL) {  
        return -1;  
    } else {  
        int lh = bstHeight(t->left);  
        int rh = bstHeight(t->right);  
        if (lh > rh) {  
            return lh + 1;  
        } else {  
            return rh + 1;  
        }  
    }  
}
```

▼ Ternary Operation Simplification

Use $1 + ((lh > rh) ? lh : rh);$



Implement the following function that returns the level of the node containing a given key if such a node exists, otherwise the function returns -1 (when a given key is not in the binary search tree).

The level of the root node is zero.

```
int bstNodeLevel(struct node *t, int key) { ... }
```

▼ Answer Recursive

```
int bstNodeLevel(struct node *t, int key) {  
    if (t == NULL) {  
        return -1;  
    } else if (t->value == key) {  
        return 0;  
    } else if (key < t->value) {  
        int level = bstNodeLevel(t->left, key);  
    } else {  
        int level = bstNodeLevel(t->right, key);  
    }  
    if (level == -1) return -1;  
    return level + 1;  
}
```

▼ Answer Iterative

```
int bstNodeLevel(structnode *t, int key) {
    int level = 0;
    struct node *curr = t;

    while (curr != NULL) {
        if (curr->value == key) {
            return level;
        } else if (key < curr->value) {
            curr = curr->left;
        } else {
            curr = curr->right;
        }

        level++;
    }

    return -1;
}
```



Implement the following function that counts the number of values that are greater than a given value in a tree.

This function should access as few nodes as possible.

```
int bstCountGreater(struct node *t, int val) { ... }
```

▼ Answer

```
int bstCountGreater(structnode *t, int val) {  
    if (t == NULL) {  
        return 0;  
    } elseif (t->value > val) {  
        return 1 + bstCountGreater(t->left, val) +  
bstCountGreater(t->right, val);  
    } else {  
        return bstCountGreater(t->right, val);  
    }  
}
```